

Agenda

- JVM Architecture
- Class Object and Reference
- Constructors
- Packages
- Access Modifiers
- Array

JVM Architecture

- 1. Compilation
 - .class file is created which consists of byte code
- 2. Byte Code
 - It is a machine level instructions that gets executed by the JVM
 - JVM converts byte code into target machine/native code
- 3. Execution
 - java is a tool used to execute the .class file.
 - It loads the .class file and invokes jvm for executing the file from the classpath
- JVM Architecture Overview
 - ClassLoader + Memory Area + Execution Engine

ClassLoader SubSystem

- It loads and initialize the class

1. Loading

- Three types of classLoaders
 - 1. Bootstrap classloader that loads built in java classes from jre/lib jars (rt.jar)
 - 2. Extension classloader that loads the extended classes from jre/lib/ext directory
 - 3. Application classloader that loads the classes from the application classpath
- It reads the classes from the disk and loads into JVM method(memory) area

2. Linking

- Three steps
 - 1. Verification : Bytecode verifier ensures that class is compiled by valid compiler and not tampered
 - 2. Preparation : Memory is allocated for static members and initialized with default values
 - 3. Resolution : Symbolic references in constant pool are replaced by the direct references

3. Initialization

- All static variables of class are assigned with their assigned values(field initializers)
- all static blocks are executed if present

Memory Areas

- There are 5 memory areas
 - 1. Method Area
 - 2. heap Area
 - 3. Stack Area
 - 4. PC Registers
 - 5. Native Method Stack Area

1. Method Area

- Create during JVM startup
- shared by all the threads
- class contents (for all classes) loaded into this area
- Method area also holds constant pool for all loaded classes.

2. Heap Area

- Create during JVM startup
- shared by all the threads
- All allocated objects (with new) are stored in heap
- The string pool is part of heap Area.
- The class Metadata is stored in a java.lang.Class object (in heap) once class is loaded.

3. Stack Area

- Separate stack is created for each thread in JVM (when thread is created).
- When a method is called a new frame (stack frame) is created on its stack.
- Each stack frame contains local variable array, operand stack, and other frame data.
- When method returns, the stack frame is destroyed.

4. PC Registers

- Separate PC register is created for each thread.
- It maintains address of the next instruction executed by the thread.
- After an instruction is completed, the address in PC is auto-incremented.

5. Native Method Stack

- Separate native method stack is created for each thread in JVM (when thread is created).
- When a native method is called from the stack, a stack frame is created on its stack.

Execution Engine

- The main component of JVM
- Convert byte code into machine code and execute it (instruction by instruction).
- It consists of
 - 1. Interpreter
 - 2. JIT Compiler
 - 3. Garbage Collector

1. Interpreter

- Each method is interpreted by the interpreter at least once.
- If method is called frequently, interpreting it each time slow down the execution of the program.
- This limitation is overcome by JIT (added in Java 1.1).

2. JIT compiler

- JIT stands for Just In Time compiler.
- Primary purpose of the JIT compiler to improve the performance.
- If a method is getting invoked multiple times, the JIT compiler convert it into native code and cache it.
- If the method is called next time, its cached native code is used to speedup execution process.

3. Profiler

- Tracks resource (memory, threads, ...) utilization for execution.
- Part of JIT that identifies hotspots. It counts number of times any method is executing.
- If the number is more than a threshold value, it is considered as hotspot.

4. Garbage Collector

- When any object is unreferenced, the GC release its memory.

JNI

JNI acts as a bridge between Java method calls and native method implementations.

Variable

- A variable is a container which holds a value.
- It represents a memory location.
- A variable is declared with data type and initialized with another variable or literal.
- In Java, variable can be
 - Local: Within a method -- Created on stack.
 - Non-static/Instance field: Within a class - Accessed using object.
 - Static field: Within a class - Accessed using class-name.

Java Method

- A method is a block of code (definition). Executes when it is called (method call).
- Method may take inputs known as parameters.
- Method may yield a output known as return value.
- Method is a logical set of instructions and can be called multiple times (reusability).
- Functions in C/CPP are called as Method in java.

Class, Reference and Object

class

- Logical entity
- blueprint of an object
- consists of fields and methods

- it is a reference type in java

Object

- physical entity
- instance of the class
- It defines three things
 - 1. state
 - 2. Behaviour
 - 3. identity
- the objects are created using new operator
- objects of the class are always created on heap.
- process of creating object of the class is also called as instantiation

Reference

- class is a reference type in java.
- variable created of a class is called as reference.
- reference variables get space on the stack.
- reference variables stores the address

Points to remember

- in java all local variables need to be initialized before using
- class fields are assigned with the default values
- primitive types are assigned 0 while references are assigned null

this Reference

- "this" is implicit reference variable that is available in every non-static method of class which is used to store reference of current/calling instance
- Whenever any non-static method is called on any object, that object is internally passed to the method and internally collected in implicit "this"
- "this" is constant within method i.e. it cannot be assigned to another object or null within the method.
- Using "this" inside method (to access members) is optional.
- However, it is good practice for readability.
- In a few cases using "this" is necessary.

Types of Methods

1. constructor
2. setters
 - Used to set value of the field from outside the class.
 - It modifies state of the object.

3. getters

- Used to get value of the field outside the class.

4. facilitators

- Provides additional functionalities
- Business logic methods

Constructor

- It is a special method of the class
- In Java fields have default values if uninitialized
- Primitive types default value is usually zero
- Reference type default value is null
- Constructor should initialize fields to the desired values.
- Types of Constructor
 - 1. Default/Parameterless Ctor
 - 2. Parameterized Ctor

Packages

- Packages makes Java code modular. It does better organization of the code.
- Package is a container that is used to group logically related classes, interfaces, enums, and other packages.
- Package helps developer:
 - To group functionally related types together.
 - To avoid naming clashing/collision/conflict/ambiguity in source code.
 - To control the access to types.
 - To make easier to lookup classes in Java source code/docs.
- Java has many built-in packages.
 - java.lang -> Integer, String, System
 - java.util -> Scanner
 - java.io -> PrintStream, Console
 - java.sql -> Connection, Statement
- To define a type inside package, it is mandatory write package declaration statement inside .java file.
- Package declaration statement must be first statement in .java file.
- Types inside package called as packaged types; while others (in default package) are unpackaged types.
- Any type can be member of single package only.

- It is standard practice to have multi-level packages (instead of single level). Typically package name is module name, dept/project name, website name in reverse order.
- When compiled, packages are created in form of directories (and sub-directories).

```
package com.sunbeaminfo.kdac
```

Access Modifiers

- For class
 - 1. default
 - 2. public
- For class members
 - 1. private
 - only within the class directly
 - 2. default (package level private)
 - in same class directly
 - in all the classes in the same package on class object
 - 3. protected
 - in same class directly
 - in all the classes in the same package on class object
 - in subclasses directly
 - 4. public
 - are visible every where.

Access Modifier	In same Package			In Different Package	
	Same class	Other Class	Sub Class	Other class	Sub Class
private	A	NA	NA	NA	NA
default	A	A	A	NA	NA
protected	A	A	A	NA	A
public	A	A	A	A	A

Difference between protected and default

- Default access restricts visibility to only classes within the same package. This allows you to encapsulate implementation details that are not intended to be accessed by classes outside the package.
- Protected access, on the other hand, allows access by subclasses (regardless of package) and by other classes within the same package.
- If you want to hide implementation details from all classes, including subclasses, default access provides stricter encapsulation.